

Lab 3: Architectural Justification

System: Self-Service Coffee Kiosk

Architecture Selection: We chose the Layered Architecture for the Self-Service Coffee Kiosk System.

Reason 1: Clear separation between UI, order logic, and hardware/data concerns

The kiosk has three distinct concerns that change independently: the touch-screen presentation, the order/payment business logic, and the storage of menu and pricing data. A Layered Architecture (Presentation, Business, Data) maps directly onto these responsibilities, so the touch-screen UI can be redesigned or restyled without touching the Order Manager, and menu pricing can be updated in the Database layer without affecting how orders are processed. For a single-purpose kiosk with a fixed, well-understood workflow (select drink, select size, pay, print receipt), this clean separation is easier to build, test, and maintain than distributing the same logic across many independent services.

Reason 2: Matches the scale and deployment needs of a single kiosk

Each kiosk is a single, self-contained unit of hardware (touch screen, card reader, receipt printer) running one application, not a distributed system serving many independent clients at scale. A Microservices approach would add unnecessary operational complexity (service discovery, network calls, independent deployments) for a device that only needs to talk to its own local hardware and a local/shared menu database. A Client-Server split, meanwhile, would introduce network dependency and a single point of failure for core in-kiosk actions such as printing a receipt. The Layered Architecture lets the whole kiosk run as one deployable application while still keeping internal responsibilities cleanly separated.

Security Advantage

Because the Presentation Layer cannot call the Payment Service or Database directly, all card-payment requests must pass through the Order Manager in the Business Layer, which acts as a single, controlled checkpoint. This means credit card data and validation logic never need to be exposed to the touch-screen UI code, reducing the attack surface and making it easier to enforce PCI-style handling rules (e.g., never storing raw card data) in one well-audited component rather than across the whole system.

Performance Benefit

Since all layers run within the same local application rather than communicating over a network (as Microservices or Client-Server would require), calls between the Order Manager, Payment Service, Receipt Printer, and Database happen as fast in-process/local calls. This keeps the order-to-payment-to-receipt flow responsive for the customer standing at the kiosk, which matters for a fast-moving café environment where checkout speed is a key usability requirement.